10 Module 10: Virtual Memory

10.1 Basic Concepts

10.1.1 Introduction to Demand Paging

- **Demand Paging** is a memory management technique where pages are loaded into main memory **only when they are needed** (on-demand), rather than loading an entire process at once.
- **Key Advantage**: Reduces swap time and physical memory requirements by avoiding the loading of unused pages.

10.1.1.1 Role of the Pager

- The **pager** (a component of the OS) makes an **educated guess** about which pages will be required immediately when a process is swapped in.
- Instead of loading the entire process, the pager **selectively loads only the necessary pages** into main memory.
- This approach optimizes memory usage and improves system performance.

10.1.2 Hardware Support for Demand Paging

Demand paging requires **hardware assistance** to distinguish between pages in memory and those on disk.

10.1.2.1 Valid-Invalid Bit

- A **valid-invalid bit** is associated with each page table entry (PTE).
  - **Valid (1)**: The page is **legal** (belongs to the process's logical address space) **and** is currently **in main memory**.
  - **Invalid (0)**: The page is either:
    - **Not valid** (does not belong to the process's logical address space), **or**
    - **Valid but on disk** (not currently in main memory).
- When a page is loaded into memory, its **page table entry is updated** to mark it as valid.
- If a page is **not in memory**, its PTE is either:
  - Marked as **invalid**, or
  - Contains the **disk address** where the page is stored.

10.1.2.1.1 Example Scenario

- Suppose only **three pages** (at frames **4, 6, and 9**) are in main memory.
  - Their **valid-invalid bits are set to valid (1)**.
- All other frames have their **valid-invalid bits set to invalid (0)**.

10.1.3 Secondary Memory (Swap Space)

- Demand paging requires **secondary storage** (typically a **high-speed disk**) to hold pages not present in main memory.
  - This storage is called the **swap device**.
  - The dedicated disk section used for demand paging is called the **swap space**.

10.1.4 Algorithms for Demand Paging

Two critical algorithms are required for efficient demand paging:

10.1.4.1 1. Frame Allocation Algorithm

Determines **how many frames** to allocate to each process and manages these allocations to: - **Optimize performance** - **Minimize page faults**

10.1.4.1.1 Common Frame Allocation Algorithms

1. **Equal Allocation**
   - Each process receives an **equal number of frames**.
   - **Pros**: Simple to implement.
   - **Cons**:
     - Does **not** consider process size or memory needs.
     - Leads to **inefficient memory usage** (small processes may get excess frames, large processes may starve).
2. **Proportional Allocation**
   - Frames are allocated **based on process size** (larger processes get more frames).
   - **Pros**: More efficient than equal allocation (accounts for process size).
   - **Cons**:
     - Still inefficient if processes have **varying page fault rates** (a large process may not need as many frames if it has low activity).
3. **Priority Allocation**
   - Frames are allocated **based on process priority** (higher-priority processes get more frames).
   - **Pros**: Ensures **critical processes** have sufficient resources.
   - **Cons**:
     - **Lower-priority processes** may experience **higher page fault rates**.

10.1.4.2 2. Page Replacement Algorithm

Handles the **selection of a victim frame** when no free frames are available.

10.1.4.2.1 Page Fault Handling Process

When a process references a page **not in memory**, the following steps occur: 1. The **processor checks the page table** and finds the entry **marked invalid**. 2. A **trap (page fault)** is triggered, transferring control to the **OS**. 3. The OS: - **Locates the desired page on disk**. - **Finds a free frame**: - If a free frame exists, it is used. - If **no free frame exists**, a **page replacement algorithm** selects a **victim frame**. 4. The **victim frame** is **written to disk** (if modified). 5. The **page and frame tables** are **updated** to reflect changes. 6. The **desired page** is **loaded into the freed frame**. 7. The **page and frame tables** are **updated again**. 8. The **user process is restarted** from the point of the page fault.

10.1.4.2.2 Overhead in Page Replacement

- When **no frames are free**, the OS must:
  1. **Swap out a victim page** (if modified).
  2. **Swap in the requested page**.
- This results in **two page transfers**, increasing overhead.

10.1.4.2.3 Minimizing Overhead with the Dirty Bit

- **Dirty Bit (Modify Bit)**:
  - A **hardware bit** that indicates whether a page has been **modified** since being loaded into memory.
  - **Set to 1** if the page is modified.
  - **Set to 0** if the page remains unchanged.
- **Optimization**:
  - Before swapping out a page, the OS **checks the dirty bit**.

- **Only modified pages (dirty bit = 1)** are written to disk.
- **Unmodified pages (dirty bit = 0)** can be **discarded without writing**, reducing I/O overhead.

10.1.5 Summary of Key Concepts

1. **Demand Paging Optimization**:
   - The pager **selectively loads pages**, reducing swap time and memory usage.
   - Improves **system performance** through efficient page management.
2. **Hardware Support**:
   - **Valid-invalid bit**: Distinguishes between pages in memory and on disk.
   - **Dirty bit**: Reduces overhead by avoiding unnecessary page writes.
3. **Algorithms**:
   - **Frame Allocation**: Determines how frames are distributed among processes (equal, proportional, or priority-based).
   - **Page Replacement**: Selects victim frames when memory is full, ensuring efficient swapping.
4. **Swap Space**:
   - Secondary storage (high-speed disk) holds pages not in main memory.
5. **Overhead Reduction**:
   - Using the **dirty bit** minimizes page transfer overhead by skipping writes for unmodified pages.

10.2 FIFO Algorithm

10.2.1 1. Introduction to FIFO Algorithm

- The **FIFO (First-In-First-Out) replacement algorithm** is one of the **earliest and simplest** page replacement strategies used in **virtual memory management**.
- It operates on the principle that the **page that has been in memory the longest** is the **first to be replaced** when a new page needs to be loaded.

10.2.2 2. Core Principle of FIFO

- **First-In-First-Out (FIFO)** means:
  - Pages are **loaded into memory in a sequence**.
  - When a **page fault** occurs (i.e., a requested page is not in memory), the **oldest page** (the one that arrived first) is **evicted** to make space for the new page.
- **No consideration of page usage frequency**-only the **order of arrival** matters.

10.2.3 3. Implementation of FIFO Algorithm

10.2.3.1 3.1 Data Structure Used

- Pages are managed using a **queue (FIFO queue)**:
  - When a page is **loaded into memory**, it is **added to the back of the queue**.
  - When a **replacement is needed**, the **page at the front of the queue** is **removed (evicted)**.

10.2.3.2 3.2 Key Components

- **Page Reference String**: A sequence of page numbers representing memory access requests.
- **Physical Memory Frames**: Fixed number of slots (e.g., 3 frames) where pages are stored.
- **Time Stamp**: Each page is tagged with the **time it was loaded** to determine replacement order.

10.2.4 4. Step-by-Step Example of FIFO Execution

10.2.4.1 Assumptions:

- **Number of frames (physical memory slots)**: 3 (F0, F1, F2)
- **Page reference string**: `1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 1, 2`
- **Initial state**: All frames are **empty**.

10.2.4.2 Execution Steps:

Time (t)	Requested Page	Frame 0 (F0)	Frame 1 (F1)	Frame 2 (F2)	Action	Page Fault?
0	1	10	-	-	Load **1** into F0	**Miss**
1	2	10	21	-	Load **2** into F1	**Miss**
2	3	10	21	32	Load **3** into F2	**Miss**
3	4	**43**	21	32	Replace **10** (oldest) with **4**	**Miss**
4	1	43	**14**	32	Replace **21** (oldest) with **1**	**Miss**
5	2	43	14	**25**	Replace **32** (oldest) with **2**	**Miss**
6	5	**56**	14	25	Replace **43** (oldest) with **5**	**Miss**
7	1	56	14	25	**1** is already in F1 (**Hit**)	**No**
8	2	56	14	25	**2** is already in F2 (**Hit**)	**No**
9	3	**39**	14	25	Replace **56** (oldest) with **3**	**Miss**
10	1	39	**110**	25	Replace **14** (oldest) with **1**	**Miss**
11	2	39	110	**211**	Replace **25** (oldest) with **2**	**Miss**

10.2.4.3 Results:

- **Total page references**: 12
- **Total page faults (misses)**: 10
- **Page fault ratio**: **10/12 ≈ 83.3%**

10.2.5 5. Advantages of FIFO Algorithm

1. **Simplicity**:
   - Easy to **understand and implement**.
   - Requires **minimal bookkeeping** (only a queue and timestamps).
2. **Predictability**:
   - Behavior is **deterministic**-replacement is based solely on **arrival time**.
   - Useful in **controlled environments** where simplicity is prioritized over performance.
3. **Low Overhead**:
   - **Computationally inexpensive** compared to more complex algorithms (e.g., LRU, Optimal).

10.2.6 6. Disadvantages of FIFO Algorithm

10.2.6.1 6.1 Lack of Adaptability

- **Does not consider page usage frequency**:
  - A **frequently accessed page** may be **evicted** simply because it was loaded early.
  - Leads to **higher page fault rates** in real-world scenarios.

10.2.6.2 6.2 Belady's Anomaly

- **Definition**: A phenomenon where **increasing the number of frames** (memory slots) **increases the number of page faults** (instead of decreasing them).
- **Implication**:
  - Normally, more frames should **reduce page faults**, but FIFO can **violate this expectation**.
  - Example:
    - With **3 frames**, FIFO may have **10 page faults** (as in the example).
    - With **4 frames**, it might **paradoxically have more faults** due to replacement order.
- **Graphical Illustration** (as mentioned in lecture):
  - A plot showing **page faults vs. number of frames** where the curve **rises unexpectedly** for FIFO.

10.2.7 7. Comparison with Other Page Replacement Algorithms

Algorithm	Replacement Criteria	Advantages	Disadvantages
FIFO	Oldest page first	Simple, low overhead	Belady's anomaly, ignores usage frequency
LRU (Least Recently Used)	Least recently accessed page	Better performance, adapts to usage	Higher overhead (requires tracking access times)
Optimal	Page not used for the longest future time	Theoretically best performance	Impossible to implement (requires future knowledge)

10.2.8 8. Practical Implications of FIFO

- **When to Use FIFO?**
  - **Embedded systems** or **real-time systems** where **simplicity and predictability** are more important than performance.
  - **Educational purposes** (to introduce page replacement concepts).
- **When to Avoid FIFO?**
  - **General-purpose operating systems** where **performance optimization** is critical.
  - Systems where **Belady's anomaly** could lead to **counterintuitive behavior**.

10.2.9 9. Summary of Key Takeaways

1. **FIFO is a simple, queue-based page replacement algorithm** that evicts the **oldest page first**.
2. **Advantages**:
   - Easy to implement.
   - Low computational overhead.
   - Predictable behavior.
3. **Disadvantages**:
   - **Ignores page usage patterns**, leading to **suboptimal performance**.

- **Belady's anomaly**: More frames can **increase page faults**.

4. **Performance**:
   - **Not suitable for high-performance systems** but useful in **controlled or educational settings**.
5. **Example Analysis**:
   - For a **12-page reference string**, FIFO resulted in **10 page faults (83.3% fault rate)**.

10.2.10 10. Conclusion

- The **FIFO algorithm** provides a **foundational understanding** of page replacement but is **not optimal** for most real-world applications due to its **lack of adaptability** and **Belady's anomaly**.
- More advanced algorithms (e.g., **LRU, LFU, Optimal**) are preferred in modern operating systems for **better performance**.
- Despite its limitations, FIFO remains **important for theoretical study** and **simplified implementations**.

10.3 Introduction to Replacement Algorithms

10.3.1 1. Introduction to Replacement Algorithms in Virtual Memory

10.3.1.1 1.1 Overview of Virtual Memory

- **Virtual memory** is a fundamental component of modern operating systems.
- It **extends physical memory (RAM)** by using **disk storage** as an auxiliary space.
- Enables systems to run larger applications or multiple processes simultaneously by **swapping data** between RAM and disk.

10.3.1.2 1.2 Role of Replacement Algorithms

- Replacement algorithms are **critical** for managing virtual memory efficiently.
- They determine **which pages (memory blocks) should be kept in physical memory** and **which should be swapped out** to disk when memory is full.
- Goal: **Optimize system performance** by minimizing unnecessary data transfers.

10.3.2 2. Key Objectives of Replacement Algorithms

Replacement algorithms serve multiple essential functions in virtual memory systems:

10.3.2.1 2.1 Efficient Utilization of Main Memory

- **Physical memory (RAM) is a limited resource**.
- Virtual memory **extends available memory** by using disk storage, but **accessing disk is slower** than RAM.
- Replacement algorithms ensure that **only the most relevant pages** remain in physical memory, maximizing efficiency.

10.3.2.2 2.2 Minimizing Page Faults

- **Page fault**: Occurs when a program accesses data **not currently in physical memory**, requiring retrieval from disk.
  - **Disk access is significantly slower** (~milliseconds) compared to RAM (~nanoseconds).
- Replacement algorithms **reduce page faults** by:
  - **Predicting which pages are least likely to be needed soon**.
  - **Swapping out (evicting) those pages** to make room for new ones.

10.3.2.3 2.3 Performance Optimization

- **Frequently accessed pages** should remain in RAM to **reduce latency**.
- **Less frequently used pages** can be swapped out to disk.
- Balancing **memory usage** and **system speed** is crucial for overall performance.

10.3.2.4 2.4 Balancing Workload in Multitasking Environments

- In **multitasking systems**, multiple processes compete for memory.
- Replacement algorithms ensure:
  - **Fair memory allocation** among processes.
  - **No single process monopolizes memory**, causing others to suffer from excessive page faults.

10.3.2.5 2.5 Handling Variable Memory Demands

- Applications have **dynamic memory requirements** (e.g., a program may need more memory at certain times).
- Replacement algorithms **dynamically adapt** to changing demands:
  - **Allocates more memory** when a process requires it.
  - **Reclaims memory** when it is no longer needed.
- Ensures **automatic adjustment** without manual intervention.

10.3.3 3. Common Replacement Algorithms

Several algorithms exist, each with different trade-offs between **simplicity**, **performance**, and **implementation complexity**.

10.3.3.1 3.1 First-In-First-Out (FIFO) Replacement

- **Definition**: Replaces the **oldest page** in memory (the one that has been in RAM the longest).
- **Mechanism**:
  - Maintains a **queue** of pages in memory.
  - When a new page must be loaded, the **first page in the queue (oldest)** is evicted.
- **Advantages**:
  - **Simple to implement** (requires only a FIFO queue).
  - **Low overhead** (minimal bookkeeping).
- **Disadvantages**:
  - **Does not consider usage patterns** (may evict frequently used pages if they are old).
  - **Can lead to higher page fault rates** compared to more sophisticated algorithms.
- **Example**:
  - If pages A, B, C enter in order, and a new page D arrives, **A is evicted** (even if it is still in use).

10.3.3.2 3.2 Least Recently Used (LRU) Replacement

- **Definition**: Replaces the **page that has not been used for the longest time**.
- **Assumption**: Pages **used recently** are **more likely to be used again soon** (temporal locality).
- **Mechanism**:
  - Requires **tracking the order of page accesses** (e.g., using a **linked list** or **timestamp**).
  - When a page is accessed, it is **moved to the front (most recently used)**.
  - The **least recently used page (tail of the list)** is evicted when needed.
- **Advantages**:
  - **Better performance** than FIFO in most cases (adapts to usage patterns).

  - **Reduces page faults** by keeping frequently used pages in memory.
- **Disadvantages**:
  - **Complex to implement** (requires maintaining access order).
  - **Higher overhead** due to frequent updates.
- **Example**:
  - If pages A, B, C are accessed in order A -> B -> C -> A, and a new page D arrives, **B is evicted** (least recently used).

10.3.3.3 3.3 Optimal (OPT) Page Replacement

- **Definition**: Replaces the **page that will not be used for the longest time in the future** (also called **Bélády's algorithm**).
- **Assumption**: **Perfect knowledge** of future memory access patterns.
- **Mechanism**:
  - For each page in memory, determine **when it will next be accessed**.
  - Evict the page with the **farthest next use** (or no future use).
- **Advantages**:
  - **Theoretically optimal** (minimizes page faults).
  - Serves as a **benchmark** for comparing other algorithms.
- **Disadvantages**:
  - **Impractical in real systems** (future access patterns are unknown).
  - **Cannot be implemented** without prior knowledge of memory requests.
- **Example**:
  - If pages A, B, C will be accessed next in order B -> C -> A, and a new page D arrives, **A is evicted** (used farthest in the future).

10.3.4 4. Summary and Key Takeaways

10.3.4.1 4.1 Importance of Replacement Algorithms

- **Critical for virtual memory efficiency**:
  - Maximizes **physical memory usage**.
  - Minimizes **page faults** (reducing disk I/O bottlenecks).
  - Balances **performance** and **resource allocation** in multitasking.
  - Adapts to **dynamic memory demands**.

10.3.4.2 4.2 Comparison of Algorithms

Algorithm	Description	Advantages	Disadvantages	Practicality
FIFO	Evicts oldest page	Simple, low overhead	Ignores usage patterns, higher page faults	High
LRU	Evicts least recently used page	Better performance, adapts to usage	Complex implementation, higher overhead	Medium
OPT	Evicts page with farthest next use	Theoretically optimal	Requires future knowledge, impractical	Low (theoretical only)

10.3.4.3 4.3 Choosing the Right Algorithm

- **No one-size-fits-all solution**:

  - **FIFO** may suffice for **simple systems** with low memory pressure.
  - **LRU** is preferred in **general-purpose OS** due to better performance.
  - **OPT** is used only for **theoretical analysis** (e.g., evaluating other algorithms).
- **Selection depends on**:
  - **System requirements** (e.g., real-time vs. general-purpose).
  - **Workload patterns** (e.g., sequential vs. random access).
  - **Implementation complexity** (e.g., hardware support for tracking page usage).

10.3.4.4 4.4 Final Remarks

- Replacement algorithms are **essential for modern OS performance**.
- **Trade-offs** exist between **simplicity**, **efficiency**, and **overhead**.
- Future advancements may introduce **hybrid or adaptive algorithms** for better optimization.

10.4 Introduction

10.4.1 1. Overview of Virtual Memory and Paging

- **Paging** is a traditional memory management technique where:
  - The **entire program** is loaded into **main memory** before execution.
  - The program is divided into fixed-size blocks called **pages**.
  - Main memory is divided into fixed-size blocks called **frames**.
  - Pages are mapped to frames for execution.
- **Key Limitation of Traditional Paging**:
  - **Inefficiency**: The entire program is loaded into memory, even if only a subset of the program is needed initially.
  - **Memory Wastage**: Unused pages occupy physical memory, increasing the **memory footprint**.

10.4.2 2. Introduction to Virtual Memory

- **Virtual Memory** is an alternative approach that addresses the inefficiencies of traditional paging.
- **Core Idea**:
  - Only **necessary portions** of a program are loaded into **main memory** when required.
  - The rest of the program remains in **secondary memory** (e.g., disk) until needed.
- **Advantages**:
  - **Reduced memory usage**: Only actively used pages consume physical memory.
  - **Efficient resource allocation**: Enables running larger programs than the available physical memory.
  - **Improved system performance**: Minimizes unnecessary memory occupation.

10.4.3 3. Demand Paging: Definition and Mechanism

10.4.3.1 3.1 Definition

- **Demand Paging** is a **virtual memory technique** where:
  - Pages of a program are **loaded into main memory only when they are accessed** (i.e., "on demand").
  - Pages that are **never accessed** remain in secondary memory and are **never loaded** into physical memory.

10.4.3.2 3.2 How Demand Paging Works

1. **Initial State**:
   - When an executable program is created, its **pages reside in secondary memory** (e.g., disk).
   - **No pages** are initially loaded into **physical memory**.

2. **Page Access Trigger**:
    - When the **CPU references a page** (e.g., during instruction execution or data access), a **page fault** occurs if the page is not in memory.
    - The **operating system (OS)** then loads the required page into a **free frame** in physical memory.
3. **Execution Continues**:
    - Once the page is loaded, the program resumes execution.

10.4.3.3 3.3 Key Terminology

Term	Definition
Demand Paging	Loading pages into memory only when they are referenced.
Page Fault	An interrupt triggered when a referenced page is not in physical memory.
Pager	A component of the OS that manages **individual pages** (as opposed to a swapper, which handles entire processes).
Lazy Swapper	An alternate name for the **pager**, emphasizing that pages are loaded **lazily** (only when needed).
Swapper	A traditional OS component that **swaps entire processes** in and out of memory (used in non-demand paging systems).

10.4.4 4. Comparison: Traditional Paging vs. Demand Paging

Feature	Traditional Paging	Demand Paging
Loading Mechanism	Entire program loaded into memory at once.	Only necessary pages loaded on demand.
Memory Usage	High (all pages occupy memory).	Optimized (only active pages in memory).
Performance Impact	Slower due to unnecessary page loading.	Faster due to reduced memory overhead.
Memory Footprint	Large (wastes memory on unused pages).	Small (minimizes unused memory occupation).
OS Component	Uses a **swapper** (handles entire processes).	Uses a **pager** (handles individual pages).

10.4.5 5. Visual Representation of Demand Paging

- **Diagram Explanation**:
    - **Secondary Memory (Right Side)**:
        - Contains **Process A** and **Process B**, each with multiple pages (e.g., A1, A2, B1, B2).
    - **Physical Memory (Left Side)**:
        - Only **required pages** (e.g., A1, B2) are loaded into **frames**.
        - Unreferenced pages (e.g., A2, B1) remain in secondary memory.

10.4.6 6. Handling Page Faults and Page Replacement

10.4.6.1 6.1 Scenario: No Free Frames Available

- If the **CPU requests a new page** but **no free frames** exist in physical memory:

1. The **OS must free up space** by **swapping out** an existing page.
2. A **page replacement algorithm** determines which page to remove.
3. The **new page** is then loaded into the freed frame.

10.4.6.2 6.2 Role of Page Replacement Algorithms

- The OS uses **page replacement policies** to decide which page to evict, such as:
  - **FIFO (First-In-First-Out)**: Evicts the oldest page in memory.
  - **LRU (Least Recently Used)**: Evicts the page that has not been used for the longest time.
  - **Optimal Algorithm**: Evicts the page that will not be used for the longest time in the future (theoretical, used as a benchmark).
  - **Clock Algorithm**: A practical approximation of LRU.

10.4.7 7. Benefits of Demand Paging

1. **Memory Efficiency**:
   - Only **actively used pages** occupy physical memory.
   - Reduces **memory footprint**, allowing more processes to run concurrently.
2. **Performance Optimization**:
   - Avoids unnecessary I/O operations (loading unused pages).
   - Improves **system responsiveness** by prioritizing essential pages.
3. **Scalability**:
   - Enables execution of **large programs** that exceed physical memory capacity.
   - Supports **multitasking** by efficiently managing memory among multiple processes.
4. **Intelligent Resource Management**:
   - Adapts to **program behavior** (e.g., loading pages only when accessed).
   - Minimizes **wasted memory** on unused code or data.

10.4.8 8. Real-World Implications

- **Modern Operating Systems** (e.g., Linux, Windows, macOS) extensively use **demand paging** to:
  - Run **memory-intensive applications** (e.g., databases, virtual machines).
  - Support **multitasking** without excessive memory consumption.
  - Improve **system stability** by preventing memory overload.
- **Growing Program Complexity**:
  - As software becomes **larger and more complex**, demand paging ensures **efficient memory usage** without requiring proportional increases in physical RAM.

10.4.9 9. Summary of Key Concepts

- **Demand paging** is a **virtual memory technique** that loads pages **only when needed**, improving memory efficiency.
- Unlike **traditional paging**, it avoids loading the **entire program** into memory, reducing waste.
- The **pager (lazy swapper)** manages individual pages, while the **swapper** handles entire processes.
- If **no free frames** are available, the OS uses a **page replacement algorithm** to evict a page and load the new one.
- **Benefits** include **reduced memory footprint**, **better performance**, and **scalability** for large programs.

10.4.10 10. Conclusion

- Demand paging is a **fundamental concept** in modern operating systems, enabling **efficient memory management**.
- It addresses the **limitations of traditional paging** by dynamically loading pages **on demand**.
- The technique is **essential** for running **large, complex applications** in environments with **limited physical memory**.
- Future lectures will explore **page replacement algorithms**, **thrashing**, and **optimization techniques** in greater detail.

10.5 LRU Algorithm

10.5.1 1. Introduction to LRU Algorithm

10.5.1.1 1.1 Definition and Core Principle

- The **Least Recently Used (LRU)** algorithm is a **page replacement algorithm** used in **virtual memory management** to determine which page should be evicted when a new page must be loaded into a full memory frame.
- **Core Idea**:
  - Pages that have been **recently accessed** are **more likely to be accessed again soon** (temporal locality principle).
  - Pages that have **not been used for the longest time** are **less likely to be needed in the near future** and are thus prime candidates for replacement.

10.5.1.2 1.2 Key Mechanism

- LRU **tracks the order in which pages are accessed** (not just loaded).
- When a **page fault** occurs (i.e., a requested page is not in memory), the algorithm **replaces the page that has not been accessed for the longest time**.
- This approach **minimizes page faults** by prioritizing the retention of frequently used pages.

10.5.1.3 1.3 Implementation Methods LRU can be implemented using: - **Counters**: Each page has a timestamp indicating the last time it was accessed. - **Stacks**: Pages are organized in a stack where the most recently used page is at the top, and the least recently used is at the bottom.

10.5.2 2. Comparison with FIFO Algorithm

Feature	LRU	FIFO
Replacement Basis	Replaces the **least recently used** page.	Replaces the **oldest loaded** page.
Tracking	Requires **last access time** of each page.	Requires **load time** of each page.
Adaptability	Adapts to **access patterns** (better performance).	Does not consider **usage frequency** (may evict frequently used pages).

Key Difference: - FIFO only considers **when a page was loaded**, while LRU considers **when a page was last accessed**.

10.5.3 3. Step-by-Step Example of LRU in Action

10.5.3.1 3.1 Scenario Setup

- **Physical Memory**: 3 frames (`f0`, `f1`, `f2`).
- **Page Reference String**: A sequence of page requests (e.g., `1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 1, 2`).
- **Initial State**: All frames are empty.
- **Notation**: `Pxt` = Page `x` referenced at time `t`.

10.5.3.2 3.2 Execution Timeline

Time (T)	Requested Page	Action	Frames After Operation	Page Fault?	Notes
0	1	Load `10` into `f0`.	`f0: 10`, `f1: –`, `f2: –`	Yes (Miss)	First access; all frames empty.
1	2	Load `21` into `f1`.	`f0: 10`, `f1: 21`, `f2: –`	Yes (Miss)	Next available frame used.
2	3	Load `32` into `f2`.	`f0: 10`, `f1: 21`, `f2: 32`	Yes (Miss)	Last empty frame filled.
3	4	Replace `10` (LRU) with `43`.	`f0: 43`, `f1: 21`, `f2: 32`	Yes (Miss)	`10` has the oldest timestamp (`t=0`).
4	1	Replace `21` (LRU) with `14`.	`f0: 43`, `f1: 14`, `f2: 32`	Yes (Miss)	`21` (`t=1`) is older than `32` (`t=2`) and `43` (`t=3`).
5	2	Replace `32` (LRU) with `25`.	`f0: 43`, `f1: 14`, `f2: 25`	Yes (Miss)	`32` (`t=2`) is older than `43` (`t=3`) and `14` (`t=4`).
6	5	Replace `43` (LRU) with `56`.	`f0: 56`, `f1: 14`, `f2: 25`	Yes (Miss)	`43` (`t=3`) is older than `14` (`t=4`) and `25` (`t=5`).
7	1	**Hit** (no replacement). Update timestamp: `14` -> `17`.	`f0: 56`, `f1: 17`, `f2: 25`	No (Hit)	Page `1` is already in `f1`; timestamp updated to reflect recent access.
8	2	**Hit** (no replacement). Update timestamp: `25` -> `28`.	`f0: 56`, `f1: 17`, `f2: 28`	No (Hit)	Page `2` is in `f2`; timestamp updated.
9	3	Replace `56` (LRU) with `39`.	`f0: 39`, `f1: 17`, `f2: 28`	Yes (Miss)	`56` (`t=6`) is older than `17` (`t=7`) and `28` (`t=8`).
10	1	**Hit** (no replacement). Update timestamp: `17` -> `110`.	`f0: 39`, `f1: 110`, `f2: 28`	No (Hit)	Page `1` is in `f1`; timestamp updated.
11	2	**Hit** (no replacement). Update timestamp: `28` -> `211`.	`f0: 39`, `f1: 110`, `f2: 211`	No (Hit)	Page `2` is in `f2`; timestamp updated.

10.5.3.3 3.3 Summary of Results

- **Total Page References**: 12
- **Page Faults (Misses)**: 8 (`T=0,1,2,3,4,5,6,9`)
- **Hits**: 4 (`T=7,8,10,11`)
- **Page Fault Ratio**: `8/12 ≈ 66.67%`

10.5.4 4. Key Observations from the Example

1. **Timestamp Updates**:
   - Whenever a page is **accessed (hit or loaded)**, its timestamp is **updated to the current time**.
   - Example: Page `1` was referenced at `T=4` (`14`) and again at `T=7` (`17`).
2. **Replacement Logic**:
   - The page with the **oldest timestamp** is always replaced.

- Example: At `T=9`, `56` (`t=6`) is replaced because it is the least recently used compared to `17` (`t=7`) and `28` (`t=8`).

3. **Performance**:
   - LRU **adapts to access patterns**, reducing unnecessary replacements of frequently used pages.

10.5.5 5. Advantages of LRU Algorithm

1. **Efficiency**:
   - Reduces **page faults** by retaining recently used pages, which are likely to be reused.
2. **Adaptability**:
   - Dynamically adjusts to **changing access patterns** (unlike FIFO, which is rigid).
3. **Optimal for Temporal Locality**:
   - Exploits the **principle of locality**, where programs tend to access the same pages repeatedly over short periods.
4. **Widely Used**:
   - Implemented in many **operating systems** (e.g., Linux, Unix) due to its balance between complexity and performance.

10.5.6 6. Disadvantages of LRU Algorithm

1. **Implementation Complexity**:
   - Requires **additional hardware/software support** to track access timestamps (e.g., counters, linked lists).
   - More complex than **FIFO** or **Random Replacement**.
2. **Overhead**:
   - Maintaining and updating timestamps for **every memory access** can introduce **performance overhead**.
3. **Not Always Optimal**:
   - While better than FIFO, LRU is **not perfect** (e.g., it may still suffer from **thrashing** in certain workloads).
4. **Cost**:
   - Hardware-based LRU (e.g., using special bits in CPU cache) can be **expensive to implement**.

10.5.7 7. Practical Applications

- **Operating Systems**:
  - Used in **page replacement** for virtual memory management.
- **Caching Systems**:
  - Employed in **CPU caches**, **database buffering**, and **web caching** (e.g., Redis, Memcached).
- **File Systems**:
  - Some file systems use LRU-like algorithms for **buffer cache management**.

10.5.8 8. Conclusion

- The **LRU algorithm** is a **practical and efficient** solution for page replacement in virtual memory systems.
- By **tracking page usage** and replacing the **least recently used** pages, it **minimizes page faults** and adapts well to **varying access patterns**.
- While **more complex** than simpler algorithms (e.g., FIFO), its **performance benefits** make it a **preferred choice** in many modern systems.

10.6 Motivation

10.6.1 Introduction to Virtual Memory

- **Definition**: Virtual memory is a memory management technique that enables a computer to **execute processes that are not entirely loaded in the main memory (RAM)**.
- **Core Functionality**:
  - Manages **limited physical memory** by **moving data between RAM and secondary storage (disk)**.
  - Creates the **illusion of a large, contiguous memory space** for applications.
  - Ensures smooth execution of programs even when **physical memory is constrained**.

10.6.2 Multi-Programming and CPU Efficiency

10.6.2.1 Role of Multi-Programming

- **Definition**: A technique that **increases CPU efficiency** by running **multiple programs concurrently or in an interleaved manner**.
- **Implementation**:
  - Multiple programs are **kept in memory simultaneously**.
  - The CPU **switches rapidly** between them, ensuring it is **never idle**.
- **Key Metric**: **Degree of multi-programming** (number of programs in memory at a given time) determines **CPU and memory utilization efficiency**.

10.6.2.2 Increasing the Degree of Multi-Programming

- **Approach**: Load **more processes into main memory**.
- **Dependencies**:
  - **Fixed/Dynamic Partition Allocation**: Depends on **size and number of partitions**.
  - **Paging**: Depends on **number of frames and frame size**.
- **Challenge**:
  - To accommodate **larger programs**, a **huge physical memory** is required, which is **often impractical**.

10.6.3 Program Behavior and Memory Usage

10.6.3.1 Error Handling and Rarely Used Code

- **Observation**: Programs often include:
  - **Error-handling code** (e.g., for invalid inputs, hardware failures) that is **rarely executed**.
  - **Unused features** (e.g., in Microsoft PowerPoint, only ~10% of features may be used).
  - **Over-allocated data structures** (arrays, lists, tables allocated more memory than needed).
- **Implication**: A **significant portion of a program's code/data is seldom accessed**.

10.6.3.2 Thrashing in Dynamic Partition Allocation

- **Definition**: **Thrashing** occurs when the system spends **more time swapping processes in/out of memory** than executing instructions.
- **Cause**: Excessive **process swapping** due to **insufficient memory** in multi-programming environments.

10.6.3.3 Principle of Locality (Locality of Reference)

- **Definition**: Programs tend to **access memory in clustered patterns** rather than uniformly.
  - **Temporal Locality**: Recently accessed data is likely to be accessed again soon.
  - **Spatial Locality**: Data near recently accessed memory is likely to be accessed next.

- **Implication**: Only a **small portion of a program's memory is actively used at any given time**.

10.6.4 Key Questions Addressed by Virtual Memory

1. **Is it necessary to load the entire program into main memory?**
   - **Answer**: **No**.
2. **Can programs be loaded in fragments (part by part) as needed?**
   - **Answer**: **Yes**, via **virtual memory**.

10.6.5 Virtual Memory: Solution and Advantages

10.6.5.1 Definition Revisited

- Virtual memory allows **execution of processes without requiring the entire program to reside in RAM**.
- Only **necessary portions (pages/segments) are loaded on demand**.

10.6.5.2 Advantages

1. **Overcoming Physical Memory Constraints**
   - Programs are **no longer limited by available RAM**.
   - Developers can include **more features** without worrying about memory size.
2. **Efficient Memory Utilization**
   - Only **actively used parts** of a program are loaded.
   - **Reduces physical memory consumption** per process.
3. **Improved Multi-Programming**
   - More programs can **coexist in memory** simultaneously.
   - **Increases the degree of multi-programming**.
4. **Enhanced CPU Utilization and Throughput**
   - Higher multi-programming -> **better CPU usage** -> **increased system throughput**.
5. **Reduced I/O Overhead**
   - Smaller program fragments -> **less I/O required** for loading/swapping.
   - **Faster program execution** due to reduced disk access.
6. **Graceful Handling of Large Programs**
   - Enables running **programs larger than physical memory**.
   - Prevents **system crashes** due to memory exhaustion.

10.6.6 Summary of Key Takeaways

1. **Problem with Traditional Memory Allocation**:
   - Fixed/dynamic partitioning and paging **require entire programs to fit in RAM**, which is **often infeasible** for large programs.
   - **Thrashing** and **inefficient memory usage** are common issues.
2. **Motivation for Virtual Memory**:
   - **Not all program parts are needed at once** (principle of locality).
   - **Error-handling code and unused features** waste memory.
   - **Need for a mechanism to load only essential fragments**.
3. **Virtual Memory as a Solution**:
   - **Loads program parts on demand** (demand paging/segmentation).
   - **Eliminates the need for contiguous physical memory**.
4. **Benefits**:
   - **Breaks the physical memory barrier** for programs.

- **Improves system performance** (CPU utilization, throughput).
- **Reduces I/O bottlenecks** and **accelerates program execution**.

5. **Broader Impact**:
   - Enables **modern computing** where applications can be **large and feature-rich** without hardware limitations.
   - **Fundamental to operating systems** (e.g., Windows, Linux, macOS).

10.7 Optimal Algorithm

10.7.1 1. Introduction to the Optimal Page Replacement Algorithm (OPT)

- The **Optimal Page Replacement Algorithm (OPT)** is a theoretically efficient page replacement strategy used in **virtual memory management**.
- It is considered the **most efficient** algorithm in terms of minimizing **page faults** (misses).
- Despite its theoretical superiority, it is **not practically implementable** in real-world systems due to its reliance on **future knowledge** of page requests.

10.7.2 2. Core Principle of the OPT Algorithm

- The algorithm replaces the **page that will not be used for the longest period in the future**.
- This ensures that the replaced page is the one least likely to be needed soon, thereby **minimizing page faults**.
- **Key Idea:** If a page is not going to be accessed for a long time, replacing it now will not cause an immediate fault.

10.7.3 3. Advantages of the OPT Algorithm

1. **Minimizes Page Faults:**
   - By definition, OPT achieves the **lowest possible page fault rate** for any given reference string.
2. **Benchmark for Other Algorithms:**
   - Serves as an **ideal standard** against which practical algorithms (e.g., **FIFO, LRU, Clock**) are compared.
   - Helps evaluate how close real-world algorithms are to **optimal performance**.
3. **Theoretical Insight:**
   - Provides a **upper bound** on performance, helping researchers understand the **best-case scenario** for page replacement.

10.7.4 4. Disadvantages of the OPT Algorithm

1. **Impracticality Due to Future Knowledge Requirement:**
   - The algorithm requires **complete foreknowledge** of the **entire page reference string** in advance.
   - In real systems, **future page requests are unknown**, making OPT **unimplementable** in practice.
2. **Theoretical-Only Use:**
   - While useful for **comparison and analysis**, it cannot be deployed in **real-time operating systems**.

10.7.5 5. Step-by-Step Example: Applying the OPT Algorithm

10.7.5.1 Problem Setup

- **Virtual Memory Configuration:**
  - **Number of frames (physical memory slots):** 3 (`f0`, `f1`, `f2`)
  - **Initial State:** All frames are empty.

- **Page Reference String (with time steps):** | Time (t) | Page Request | |————-|—————| | 0 | 1 | | 1 | 2 | | 2 | 3 | | 3 | 4 | | 4 | 1 | | 5 | 2 | | 6 | 5 | | 7 | 1 | | 8 | 2 | | 9 | 3 | | 10 | 1 | | 11 | 2 |

10.7.5.2 Execution Trace

10.7.5.2.1 t = 0: Request for Page 1

- **Action:** Page 1 is **not in memory** (miss).
- **Result:** Load Page 1 into `f0`.
- **Frame State:** `[1, -, -]`

10.7.5.2.2 t = 1: Request for Page 2

- **Action:** Page 2 is **not in memory** (miss).
- **Result:** Load Page 2 into `f1`.
- **Frame State:** `[1, 2, -]`

10.7.5.2.3 t = 2: Request for Page 3

- **Action:** Page 3 is **not in memory** (miss).
- **Result:** Load Page 3 into `f2`.
- **Frame State:** `[1, 2, 3]`

10.7.5.2.4 t = 3: Request for Page 4

- **Action:** Page 4 is **not in memory** (miss), and **all frames are full**.
- **OPT Decision:**
  - Check **future references** of pages in memory (`1`, `2`, `3`):
    - Page 1 -> Next used at **t=4** (soon).
    - Page 2 -> Next used at **t=5** (soon).
    - Page 3 -> **Not used until t=9** (longest gap).
  - **Replace Page 3** (since it won't be needed for the longest time).
- **Result:** Replace Page 3 with Page 4.
- **Frame State:** `[1, 2, 4]`

10.7.5.2.5 t = 4: Request for Page 1

- **Action:** Page 1 is **already in memory** (hit).
- **Result:** No change.
- **Frame State:** `[1, 2, 4]`

10.7.5.2.6 t = 5: Request for Page 2

- **Action:** Page 2 is **already in memory** (hit).
- **Result:** No change.
- **Frame State:** `[1, 2, 4]`

10.7.5.2.7 t = 6: Request for Page 5

- **Action:** Page 5 is **not in memory** (miss).
- **OPT Decision:**
  - Check future references of pages in memory (`1`, `2`, `4`):

        - Page 1 -> Next used at **t=7** (soon).
        - Page 2 -> Next used at **t=8** (soon).
        - Page 4 -> **Not used again** (can be replaced safely).
    - **Replace Page 4** (since it won't be needed at all in the future).
- **Result:** Replace Page 4 with Page 5.
- **Frame State:** `[1, 2, 5]`

10.7.5.2.8 t = 7: Request for Page 1

- **Action:** Page 1 is **already in memory** (hit).
- **Result:** No change.
- **Frame State:** `[1, 2, 5]`

10.7.5.2.9 t = 8: Request for Page 2

- **Action:** Page 2 is **already in memory** (hit).
- **Result:** No change.
- **Frame State:** `[1, 2, 5]`

10.7.5.2.10 t = 9: Request for Page 3

- **Action:** Page 3 is **not in memory** (miss).
- **OPT Decision:**
    - Check future references of pages in memory (`1`, `2`, `5`):
        - Page 1 -> Next used at **t=10** (soon).
        - Page 2 -> Next used at **t=11** (soon).
        - Page 5 -> **Not used again** (can be replaced safely).
    - **Replace Page 5** (since it won't be needed in the future).
- **Result:** Replace Page 5 with Page 3.
- **Frame State:** `[1, 2, 3]`

10.7.5.2.11 t = 10: Request for Page 1

- **Action:** Page 1 is **already in memory** (hit).
- **Result:** No change.
- **Frame State:** `[1, 2, 3]`

10.7.5.2.12 t = 11: Request for Page 2

- **Action:** Page 2 is **already in memory** (hit).
- **Result:** No change.
- **Frame State:** `[1, 2, 3]`

10.7.5.3 Summary of Page Faults

- **Total Page Requests:** 12
- **Total Page Faults (Misses):** 6 (at t=0,1,2,3,6,9)
- **Page Fault Ratio: 6/12 = 0.5 (50%)**

10.7.6 6. Key Observations from the Example

1. **OPT Minimizes Faults:**
   - The algorithm **never replaces a page that will be needed soon**, ensuring the **lowest possible fault count**.
2. **Future Knowledge is Critical:**
   - Every replacement decision depends on **looking ahead** in the reference string.
3. **Comparison with Other Algorithms:**
   - Practical algorithms (e.g., **LRU, FIFO**) would likely have **more faults** for the same reference string.

10.7.7 7. Practical Implications and Use Cases

1. **Benchmarking Tool:**
   - Used to **evaluate the efficiency** of real-world algorithms by comparing their fault rates to OPT.
2. **Theoretical Research:**
   - Helps in **designing new algorithms** that approximate OPT's behavior without requiring future knowledge.
3. **Educational Value:**
   - Demonstrates the **ideal case** for page replacement, aiding in understanding **trade-offs** in memory management.

10.7.8 8. Conclusion: Why OPT Matters Despite Being Unimplementable

- **Theoretical Perfection:** Provides an **unachievable but ideal standard** for page replacement.
- **Performance Metric:** Allows researchers to **quantify how close** practical algorithms are to optimality.
- **Design Insight:** Guides the development of **better heuristic-based algorithms** (e.g., **LRU, LFU, Clock**).

10.7.8.1 Final Takeaway:

> *"While the Optimal Page Replacement Algorithm cannot be used in real systems due to its reliance on future knowledge, it remains an indispensable tool for analyzing and improving practical page replacement strategies."*

10.8 Summary

10.8.1 Introduction to Virtual Memory

- **Definition**: Virtual memory is a fundamental concept in modern computing that creates the illusion of a **large, contiguous block of memory**, allowing applications to run smoothly even when **physical memory (RAM) is constrained**.
- **Key Motivation**:
  - **Partial Program Loading**: Not all parts of a program need to be in **main memory (RAM)** at the same time.
  - **Efficient Memory Utilization**: Enables better **multiprogramming**, improving **CPU utilization** and **system throughput**.

10.8.2 Core Concepts of Virtual Memory

10.8.2.1 1. Memory Division and Address Space

- **Physical Memory (RAM)** is divided into fixed-size blocks called **frames**.
- **Virtual Address Space** (per process) is divided into fixed-size blocks called **pages**.

- **Mapping**: The **Memory Management Unit (MMU)** translates **virtual addresses** to **physical addresses** using **page tables**.

10.8.2.2 2. Role of the Pager (Lazy Swapper)

- **Definition**: A **pager** is a component of the operating system responsible for:
  - **Paging Out**: Moving pages from **main memory (RAM)** to **secondary storage (disk)** when needed.
  - **Paging In**: Loading pages back into **main memory** when accessed.
- **Alternative Name**: Also called a **"Lazy Swapper"** because it delays loading pages until absolutely necessary (demand paging).

10.8.3 Advantages of Virtual Memory

10.8.3.1 1. Improved Degree of Multiprogramming

- **Multiprogramming**: Running multiple processes simultaneously.
- **Benefit**: Virtual memory allows more processes to reside in memory (even if partially), increasing **CPU utilization** and **throughput**.

10.8.3.2 2. Large Address Space Illusion

- **Perception**: Applications perceive a **contiguous, large address space**, far exceeding the **actual physical memory** available.
- **Programmer Benefit**: Developers are **not constrained by limited RAM**; they can write programs assuming abundant memory.

10.8.4 Demand Paging

- **Definition**: A **virtual memory technique** where pages are **loaded into main memory only when they are accessed** (on-demand).
- **Key Benefits**:
  - **Reduced Initial Load Time**: Programs start faster since not all pages are loaded at once.
  - **Minimized Physical Memory Footprint**: Only necessary pages occupy RAM, freeing space for other processes.

10.8.4.1 Implementation Mechanics

1. **Page Fault Handling**:
   - If a process accesses a page **not in RAM**, a **page fault** occurs.
   - The OS fetches the required page from **disk** into a **free frame**.
   - If no free frames exist, a **page replacement algorithm** selects a victim frame to evict.
2. **Page Table Entries (PTEs)**:
   - Each virtual page has an entry in the **page table** containing:
     - **Frame number** (if the page is in RAM).
     - **Control bits** (e.g., **valid-invalid bit**, **dirty bit**).

10.8.5 Page Table Control Bits

10.8.5.1 1. Valid-Invalid Bit

- **Purpose**: Indicates whether a page is **currently in main memory**.
  - **Valid (1)**: Page is in RAM; the frame number in the PTE is valid.
  - **Invalid (0)**: Page is **not in RAM** (either on disk or not yet loaded).

- **Role in Memory Management**:
  - Prevents illegal memory access.
  - Triggers a **page fault** if an invalid page is accessed.

10.8.5.2 2. Dirty Bit (Modified Bit)

- **Purpose**: Indicates whether a page has been **modified since being loaded into RAM**.
  - **Dirty (1)**: Page has been written to; must be **saved to disk** before replacement.
  - **Clean (0)**: Page has not been modified; can be **discarded** without writing to disk.
- **Optimization**: Reduces unnecessary **disk I/O** during page replacement by avoiding writes for unmodified pages.

10.8.6 Algorithms in Virtual Memory Management

10.8.6.1 1. Frame Allocation Algorithms

- **Purpose**: Determine how **frames (physical memory blocks)** are distributed among competing processes.
- **Types Discussed**:
  1. **Equal Allocation**:
     - Each process gets an **equal number of frames**.
     - **Merit**: Simple and fair.
     - **Demerit**: Ignores process size and memory needs.
  2. **Proportional Allocation**:
     - Frames are allocated based on **process size**.
     - **Merit**: Larger processes get more frames.
     - **Demerit**: May not account for **actual memory usage patterns**.
  3. **Priority Allocation**:
     - Higher-priority processes receive **more frames**.
     - **Merit**: Supports **real-time and critical processes**.
     - **Demerit**: Lower-priority processes may suffer.

10.8.6.2 2. Page Replacement Algorithms

- **Purpose**: Select which **frame to evict** when a new page must be loaded and no free frames are available.
- **Algorithms Discussed**:

10.8.6.3 A. First-In-First-Out (FIFO)

  - **Mechanism**: Replaces the **oldest page** in memory.
  - **Merits**:
    - Simple to implement (uses a **queue**).
  - **Demerits**:
    - **Belady's Anomaly**: More frames can lead to **more page faults** (counterintuitive behavior).
    - Does not consider **page usage frequency**.

10.8.6.4 B. Least Recently Used (LRU)

  - **Mechanism**: Replaces the **page that has not been used for the longest time**.
  - **Implementation**:
    - Requires tracking **page access history** (e.g., using a **stack** or **linked list**).
  - **Merits**:

        - **Highly effective** in practice due to **locality of reference** (recently used pages are likely to be reused).
        - Avoids Belady's Anomaly.
    - **Demerits**:
        - **Overhead**: Maintaining access history can be costly.

10.8.6.5 C. Optimal Replacement (OPT / MIN)

- **Mechanism**: Replaces the page that **will not be used for the longest time in the future**.
- **Merits**:
    - **Theoretically perfect**: Minimizes page faults.
    - Used as a **benchmark** to evaluate other algorithms.
- **Demerits**:
    - **Unimplementable** in real systems (requires **future knowledge** of page accesses).
    - Only useful for **simulation and comparison**.

10.8.7 Key Observations on Algorithms

- **FIFO's Belady's Anomaly**:
    - Increasing the number of frames can **increase page faults**, which is **counterintuitive**.
    - Example: A specific page reference sequence may cause more faults with 4 frames than with 3.
- **LRU's Practicality**:
    - **Most widely used** due to its **balance between performance and implementability**.
    - Relies on the **principle of locality** (temporal and spatial).
- **Optimal Algorithm's Role**:
    - Serves as a **gold standard** for comparison but is **not feasible** in real-world systems.

10.8.8 Conclusion

- Virtual memory enables **efficient memory management** by:
    - Providing a **large, contiguous address space** illusion.
    - Reducing **memory constraints** for programmers.
    - Improving **system performance** via **demand paging** and **smart replacement policies**.
- **Critical Components**:
    - **Pager (Lazy Swapper)**: Manages page movement between RAM and disk.
    - **Page Table Bits (Valid/Invalid, Dirty)**: Ensure correct and efficient memory operations.
    - **Allocation & Replacement Algorithms**: Balance fairness, performance, and overhead.

End of Notes

10.9 Virtual Memory Concept

10.9.1 1. Introduction to Virtual Memory

- Virtual memory is a memory management technique that provides an **illusion of a large, contiguous block of memory** to processes.
- It allows **efficient memory utilization** by enabling processes to execute even when only a **portion of their code/data is in physical memory**.
- Unlike traditional paging, virtual memory **decouples logical (virtual) address space from physical address space**.

10.9.2 2. Comparison with Paging Memory Allocation

10.9.2.1 2.1 Paging Technique Recap

- **Physical memory** is divided into fixed-size blocks called **frames**.
- **Logical memory** is divided into fixed-size blocks called **pages**.
- The **CPU generates logical addresses**, which are translated into **physical addresses** by the **Memory Management Unit (MMU)**.
- **Key Limitation**:
  - The **entire program must reside in main memory** for execution.
  - **Physical memory size >= Logical address space** of the program.

10.9.2.2 2.2 Virtual Memory vs. Paging

Feature	Paging Technique	Virtual Memory Technique
Memory Requirement	Entire program must be in main memory.	Only **part of the program** needs to be in memory.
Address Space	Logical <= Physical address space.	**Logical > Physical** address space allowed.
Multi-programming	Limited by physical memory constraints.	**Enables higher degree of multi-programming** (multiple processes share physical memory).
Process Creation	Less efficient (requires full loading).	**More efficient** (partial loading possible).

10.9.3 3. Core Concepts of Virtual Memory

10.9.3.1 3.1 Virtual Address Space

- Each process is assigned a **contiguous virtual address space**, giving the illusion of a **large, dedicated memory block**.
- The **virtual address space** is **independent of physical memory size**.
- Example:
  - A process may have a **4GB virtual address space**, but the system may only have **8GB of physical RAM** shared among all processes.

10.9.3.2 3.2 Physical Memory Organization

- **Physical memory (RAM)** is divided into **frames** (fixed-size blocks).
- **Virtual memory** is divided into **pages** (same fixed size as frames).
- Only **some pages** from the virtual address space are **loaded into physical frames** at any time.

10.9.3.3 3.3 Address Translation Mechanism

- The **CPU generates virtual addresses**.
- The **Memory Management Unit (MMU)**, a hardware component, **translates virtual addresses -> physical addresses**.
- **Page Table**:
  - Maintained by the MMU for **each process**.
  - Maps **virtual pages -> physical frames**.
  - If a page is **not in physical memory**, it must be **loaded from disk** (handled by the OS).

10.9.4 4. Handling Memory Shortages: Page Replacement

10.9.4.1 4.1 Scenario: No Free Frames Available

- When the **CPU references a virtual page not in physical memory**, a **page fault** occurs.
- If **no free frames** are available, the OS must:
  1. **Select a victim page** (using a **page replacement algorithm**).
  2. **Swap out** the victim page to disk (if modified).
  3. **Load the new page** into the freed frame.
  4. **Update the page table** to reflect the change.

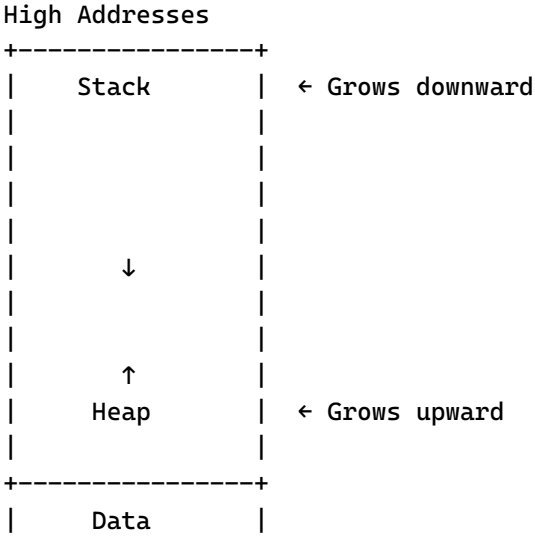
10.9.4.2 4.2 Page Replacement Algorithms

- The OS decides **which page to evict** using algorithms such as:
  - **FIFO (First-In-First-Out)**
  - **LRU (Least Recently Used)**
  - **Optimal Page Replacement** (theoretical, requires future knowledge)
  - **Clock Algorithm** (approximation of LRU)
- Goal: **Minimize page faults** while maximizing performance.

10.9.5 5. Structure of Virtual Address Space

- The virtual address space of a process is typically divided into **four segments**:
  1. **Code (Text) Segment**:
     - Contains **executable instructions** of the program.
     - Usually **read-only** (to prevent modification).
  2. **Data Segment**:
     - Stores **global and static variables**.
  3. **Heap**:
     - Used for **dynamic memory allocation** (e.g., `malloc`, `new`).
     - **Grows upward** in memory as more allocations occur.
  4. **Stack**:
     - Stores **local variables and function call frames**.
     - **Grows downward** as functions are called recursively or nested.

10.9.5.1 5.1 Visual Representation



```
+----------------+
|     Code       |
Low Addresses
```

- **Gap between Heap and Stack**:
  - Reserved for **future expansion** of either segment.
  - **Not backed by physical memory** until actually used.

10.9.6 6. Advantages of Virtual Memory

1. **Illusion of Large Memory**:
   - Processes can use **more memory than physically available** (via disk swapping).
2. **Efficient Multi-programming**:
   - Multiple processes can **share physical memory** without interference.
3. **Better Process Isolation**:
   - Each process operates in its own **virtual address space**, preventing unauthorized access.
4. **Flexible Memory Allocation**:
   - Only **required pages** are loaded, reducing memory wastage.
5. **Simplified Memory Management**:
   - Programs can use **contiguous virtual addresses** without worrying about physical fragmentation.

10.9.7 7. Key Takeaways

- Virtual memory **extends the paging concept** by allowing **logical address space > physical memory**.
- The **MMU** translates **virtual -> physical addresses** using **page tables**.
- If a referenced page is **not in memory**, a **page fault** occurs, and the OS **loads it from disk** (possibly evicting another page).
- Virtual address space is structured into **code, data, heap, and stack**, with **heap growing upward** and **stack growing downward**.
- Virtual memory enables:
  - **Higher degree of multi-programming**.
  - **More efficient process creation**.
  - **Better utilization of limited physical memory**.